



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

АВЛ дерево

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Добавление элемента	5
2.2 Удаление элемента	5
2.3 Поиск элемента	6
3 Технологическая часть	8
4 Исследовательская часть	15
Заключение	20
Список используемых источников	21

Введение

Цель лабораторной работы: разработать программу для создания АВЛ-дерева.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать структуру АВЛ-дерева.
2. Описать алгоритмы добавления и удаления элемента, поиска элемента в дереве, балансировки дерева.
3. Разработать программу, предоставляющую пользователю выбор пункта меню, отображающую меню в цикле с постусловием, реализующую все описанные алгоритмы.
4. Выполнить тестирование программы.
5. Привести примеры работы программы.

1 Аналитическая часть

АВЛ-дерево [1] — это прежде всего двоичное дерево поиска, ключи которого удовлетворяют стандартному свойству: ключ любого узла дерева не меньше любого ключа в левом поддереве данного узла и не больше любого ключа в правом поддереве этого узла. Основная идея состоит в том, чтобы поддерживать высоты левого и правого поддеревьев каждого узла в балансе так, чтобы разница высот поддеревьев для каждого узла не превышала 1 (то есть разница высот должна быть -1 , 0 или 1). Каждый узел АВЛ-дерева содержит ключ, ссылки на левое и правое поддерева, а также информацию о высоте. При вставке или удалении узлов дерево перебалансируется, чтобы сохранить свойство баланса высот. Это происходит путём поворотов узлов вокруг своих осей.

2 Конструкторская часть

2.1 Добавление элемента

1. Вставка нового элемента.

- Создать функцию “insert”, которая принимает корень дерева и новый ключ для вставки.
- Если дерево пустое (корень — “nullptr”), создать новый узел с переданным ключом и сделать его корнем дерева.
- Если ключ меньше ключа текущего узла, рекурсивно вставить его в левое поддерево.
- Если ключ больше ключа текущего узла, рекурсивно вставить его в правое поддерево.
- После вставки узла, обновить высоту текущего узла.
- Вычислить балансировку “balanceFactor” для текущего узла (разница высот левого и правого поддерева).
- Если балансировка узла нарушена (больше 1 или меньше -1), выполнить балансировку путем вращений.

2. Балансировка дерева.

- Если баланс больше 1 и вставка была в левое поддерево левого потомка, выполнить правый поворот для текущего узла.
- Если баланс меньше -1 и вставка была в правое поддерево правого потомка, выполнить левый поворот для текущего узла.
- Если баланс больше 1 и вставка была в правое поддерево левого потомка, выполнить двойной поворот. Сначала левый поворот для левого потомка, затем правый поворот для текущего узла.
- Если баланс меньше -1 и вставка была в левое поддерево правого потомка, выполнить двойной поворот. Сначала правый поворот для правого потомка, затем левый поворот для текущего узла.

2.2 Удаление элемента

1. Начало удаления.

- Функция “remove” принимает корень дерева “root” и ключ “key” для удаления.
- Если корень дерева пуст, возвращается “nullptr”, так как узла для удаления не существует.

2. Поиск узла для удаления.

- Если ключ меньше ключа текущего узла, рекурсивно вызывается “remove” для левого поддерева.
- Если ключ больше ключа текущего узла, рекурсивно вызывается “remove” для правого поддерева.
- Если ключ совпадает с ключом текущего узла, происходит удаление узла.

3. Удаление узла.

- Если у узла есть один потомок, узел заменяется этим потомком.
- Если у узла два потомка, найденный узел с наименьшим ключом в правом поддерева заменяет удаляемый узел.

4. Балансировка после удаления.

- После удаления узла, пересчитывается высота текущего узла. Проверяется баланс текущего узла.
- Если баланс больше 1 и левое поддерево текущего узла сбалансировано ($\text{balanceFactor}(\text{root-left}) \geq 0$), выполняется правый поворот.
- Если баланс больше 1 и левое поддерево текущего узла несбалансировано ($\text{balanceFactor}(\text{root-left}) < 0$), выполняется левый поворот, а затем правый поворот.
- Если баланс меньше -1 и правое поддерево текущего узла сбалансировано ($\text{balanceFactor}(\text{root-right}) \leq 0$), выполняется левый поворот.
- Если баланс меньше -1 и правое поддерево текущего узла несбалансировано ($\text{balanceFactor}(\text{root-right}) > 0$), выполняется правый поворот, а затем левый поворот.

2.3 Поиск элемента

1. Функция принимает указатель на корень дерева “root” и ключ “key” для поиска.
2. Если корень дерева пуст, возвращается “false”, так как узла с заданным ключом в дереве нет.
3. Сравнение ключа.
 - Если ключ совпадает с ключом текущего узла, возвращается “true”, так как узел с искомым ключом найден.
 - Если ключ меньше ключа текущего узла, рекурсивно вызывается “search” для левого поддерева.
 - Если ключ больше ключа текущего узла, рекурсивно вызывается “search” для правого поддерева.

4. Возврат результата.

- Если узел с искомым ключом найден, возвращается “true”.
- Если узел не найден, возвращается “false”, так как узла с искомым ключом в дереве нет.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<queue>`. Она предоставляет реализацию очереди (queue) - структуры данных, основанной на принципе "первым пришёл — первым вышел" (FIFO - First-In-First-Out). Это означает, что элементы добавляются в конец очереди, а извлекаются из начала. Библиотека `<algorithm>` в C++ предоставляет множество стандартных функций для работы с контейнерами, итераторами и другими структурами данных.

Далее приведен листинг реализации трех алгоритмов (листинг 1).

Листинг 1 – Исходный код программы для построения AVL-дерева

```
#include <iostream>
#include <algorithm>
#include <queue>

using namespace std;

class AVLTree {
private:
    struct Node {
        int key;
        Node* left;
        Node* right;
        int height;

        Node(int k) : key(k), left(nullptr), right(nullptr),
                     height(1) {}
    };

    Node* root;

    int height(Node* node) {
        if (node == nullptr) return 0;
        return node->height;
    }

    int balanceFactor(Node* node) {
        if (node == nullptr) return 0;
        return height(node->left) - height(node->right);
    }
};
```

```

Node* rightRotate(Node* root) {
    Node* x = root->left;
    Node* T2 = x->right;
    x->right = root;
    root->left = T2;

    root->height = max(height(root->left), height(root->right
        )) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

```

```

Node* leftRotate(Node* root) {
    Node* y = root->right;
    Node* T2 = y->left;
    y->left = root;
    root->right = T2;

    root->height = max(height(root->left), height(root->right
        )) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

```

```

Node* insert(Node* node, int key) {
    if (node == nullptr) return new Node(key);

    if (key < node->key) {
        node->left = insert(node->left, key);
    }
    else if (key > node->key) {
        node->right = insert(node->right, key);
    }
    else {
        return node;
    }

    node->height = 1 + max(height(node->left), height(node->
        right));
}

```

```

    int balance = balanceFactor(node);

    if (balance > 1 && key < node->left->key) {
        return rightRotate(node);
    }
    if (balance < -1 && key > node->right->key) {
        return leftRotate(node);
    }
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

Node* minValueNode(Node* node) {
    Node* current = node;

    while (current->left != nullptr) {
        current = current->left;
    }

    return current;
}

Node* remove(Node* root, int key) {
    if (root == nullptr) return root;

    if (key < root->key) {
        root->left = remove(root->left, key);
    }
    else if (key > root->key) {
        root->right = remove(root->right, key);
    }
    else {
        if (root->left == nullptr || root->right ==

```

```

        nullptr) {
            Node* temp = root->left ? root->left :
                root->right;

            if (temp == nullptr) {
                temp = root;
                root = nullptr;
            }
            else{
                *root = *temp;
            }

            delete temp;
        }
        else {
            Node* temp = minValueNode(root->right);
            root->key = temp->key;
            root->right = remove(root->right, temp->
                key);
        }
    }

    if (root == nullptr) return root;

    root->height = 1 + max(height(root->left), height(root->
        right));

    int balance = balanceFactor(root);

    if (balance > 1 && balanceFactor(root->left) >= 0) {
        return rightRotate(root);
    }
    if (balance > 1 && balanceFactor(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if (balance < -1 && balanceFactor(root->right) <= 0) {
        return leftRotate(root);
    }
    if (balance < -1 && balanceFactor(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

```

```

        }

        return root;
    }

bool search(Node* root, int key) {
    if (root == nullptr) return false;

    if (key == root->key) {
        return true;
    }
    else if (key < root->key) {
        return search(root->left, key);
    }
    else {
        return search(root->right, key);
    }
}

public:
    AVLTree() : root(nullptr) {}

    void insert(int key) {
        root = insert(root, key);
    }

    void remove(int key) {
        root = remove(root, key);
    }

    bool search(int key) {
        return search(root, key);
    }

    void printTreeLevelOrder() {
        if (root == nullptr) {
            cout << "Дерево пусто." << endl;
            return;
        }

        cout << "Дерево по( уровням):" << endl;
        queue<Node*> levelQueue;

```

```

        levelQueue.push(root);

        while (!levelQueue.empty()) {
            int levelSize = levelQueue.size();

            while (levelSize > 0) {
                Node* current = levelQueue.front();
                levelQueue.pop();

                cout << current->key << " ";

                if (current->left != nullptr) {
                    levelQueue.push(current->left);
                }
                if (current->right != nullptr) {
                    levelQueue.push(current->right);
                }

                levelSize--;
            }

            cout << endl;
        }
    }

};

int main() {
    setlocale(0, "");
    AVLTree tree;

    int choice, key;
    bool running = true;

    while (running) {
        cout << "Выберите операцию:" << endl;
        cout << "1. Добавитьэлемент " << endl;
        cout << "2. Удалитьэлемент " << endl;
        cout << "3. Поискэлемента " << endl;
        cout << "4. Вывестидеревопо ( уровням)" << endl;
        cout << "5. Выйтиизпрограммы " << endl;
        cout << "Введите выбор: ";
        cin >> choice;
    }
}

```

```

switch (choice) {
case 1:
    cout << "Введите ключдлядобавления : ";
    cin >> key;
    tree.insert(key);
    cout << endl;
    break;
case 2:
    cout << "Введите ключдляудаления : ";
    cin >> key;
    tree.remove(key);
    cout << endl;
    break;
case 3:
    cout << "Введите ключдляпоиска : ";
    cin >> key;
    if (tree.search(key))
        cout << "Элемент найден." << endl;
    else
        cout << "Элемент не найден ." << endl;
    cout << endl;
    break;
case 4:
    tree.printTreeLevelOrder();
    break;
case 5:
    running = false;
    break;
default:
    cout << "Неверный выбор! Попробуйтеещераз ." << endl;
}

}

return 0;

}

```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы.

Результат работы программы, в котором создается АВЛ-дерево путем добавления туда элементов. Далее выводится созданное дерево, удаляется элемент и снова выводится АВЛ-дерево, приведен на рис. 1.

```
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 50

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 90

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 20

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 10

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 2

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 100

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 4
Дерево (по уровням):
50
10 90
2 20 100
```

Рис. 1 – Пример №1

```
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 2
Введите ключ для удаления: 20

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 4
Дерево (по уровням):
50
10 90
2 100

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 5
```

Рис. 2 – Пример №1 (продолжение)

Создается АВЛ-дерево путем добавления туда элементов. Далее несуществующий элемент удаляется и ищется в АВЛ-дереве. Удаляется введенный элемент и выводится пустое дерево. Результат работы программы, приведен на рис. 3.

```
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 100

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 4
Дерево <по уровням>:
100

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 2
Введите ключ для удаления: 66

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 3
Введите ключ для поиска: 66
Элемент не найден.

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 2
Введите ключ для удаления: 100

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 4
Дерево пусто.

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 5
```

Рис. 3 – Пример №2

Создается АВЛ-дерево путем добавления туда одинаковых элементов. Результат работы программы, приведен на рис. 4.

```
Введите ключ для добавления: 20
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 20
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 20
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 20
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 4
Дерево <по уровням>:
20
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево <по уровням>
5. Выйти из программы
Введите выбор: 5
```

Рис. 4 – Пример №3

Создается AVL-дерево путем добавления туда элементов. Далее существующий ищется в AVL-дереве и выводится построенное AVL-дерево. Результат работы программы, приведен на рис. 5.

```
Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 50

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 1
Введите ключ для добавления: 51

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 3
Введите ключ для поиска: 51
Элемент найден.

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 4
Дерево (по уровням):
50
51

Выберите операцию:
1. Добавить элемент
2. Удалить элемент
3. Поиск элемента
4. Вывести дерево (по уровням)
5. Выйти из программы
Введите выбор: 5
```

Рис. 5 – Пример №4

Заключение

Цель лабораторной работы достигнута. Разработана программа для создания АВЛ-дерева. В результате выполнения лабораторной работы были выполнены все задачи.

1. Описана структура АВЛ-дерева.
2. Описаны алгоритмы добавления и удаления элемента, поиска элемента в дереве, балансировки дерева.
3. Разработана программа, предоставляющая пользователю выбор пункта меню, отображающая меню в цикле с постусловием, реализующая все описанные алгоритмы.
4. Выполнено тестирование программы.
5. Приведены примеры работы программы.

Список литературы

1. Никлаус Вирт. Алгоритмы и структуры данных. / Пер. с англ. Ткачев Ф. В. — М.: ДМК Пресс, 2010. — 272 с.